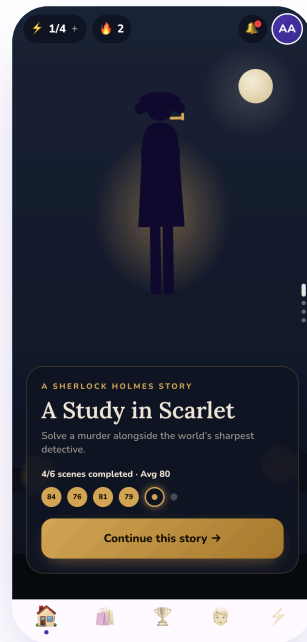


Amiboli

Developer Handoff Specification



Flutter app

FastAPI backend

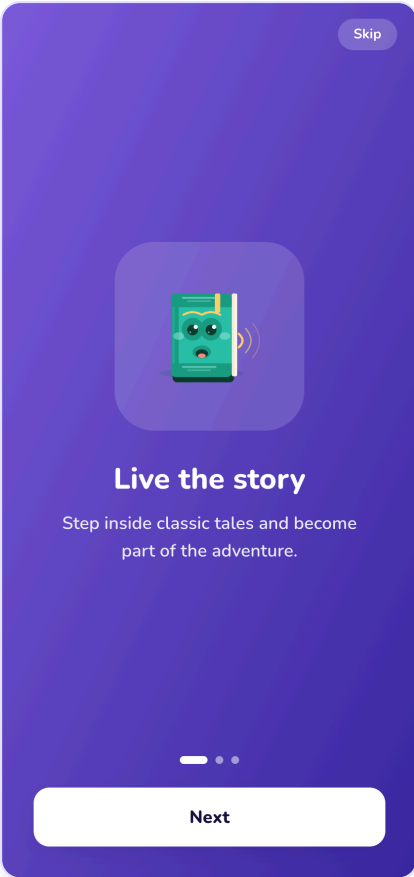
Brand v2

Build a functional app matching this prototype.
Live reference: amiboli-app-prototype.vercel.app

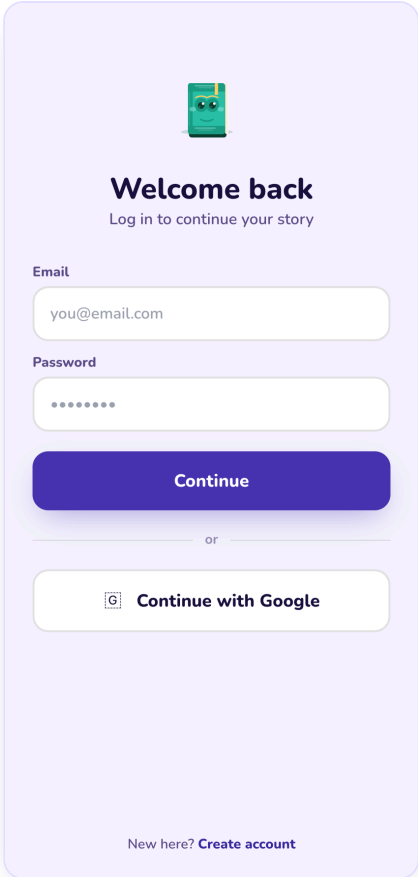
Screen gallery — what you're building



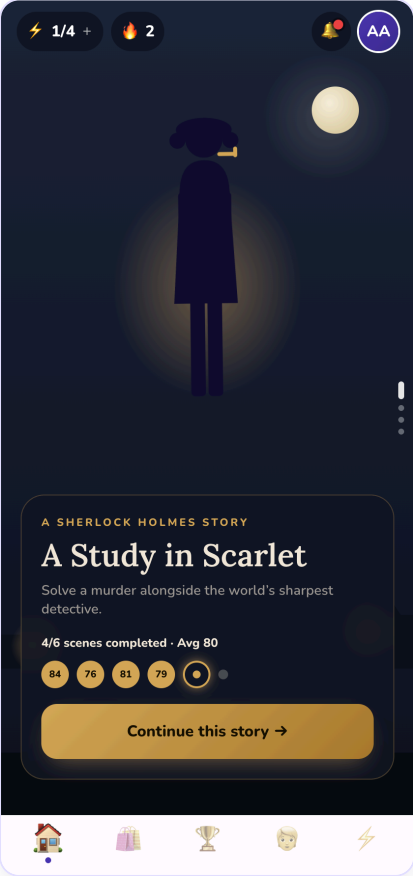
Splash
Animated Ami logo. Auto-advances to Home.



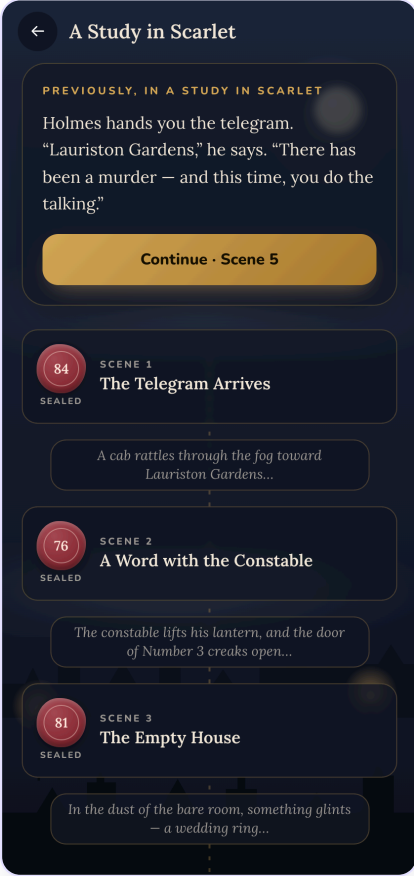
Onboarding
3 swipeable slides, skippable.



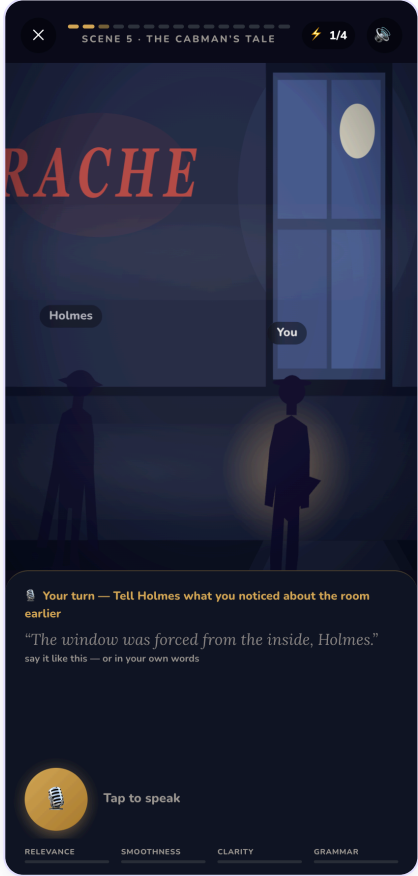
Auth (UI only)
Any tap → Home.



Home — book feed
Scroll-snap, one book per screen, big moving character.



Book detail
Recap/intro + vertical scene path.



Scene player
Header · shadow-theatre stage · caption (mic mode).

3

DAY STREAK!

80

Scene sealed

Scene 5 · The Cabman's Tale

TIME

2:48

ENERGY

-1

TURN

4

SCORE EARNED

+320

Total score

3,920 → 4,240

Streak extended to 3 days

HOW YOU SPOKE

Relevance

82

+0 vs avg

Smoothness

74

+10 vs avg

Clarity

70

+1 vs avg

Replay

Grammar

Back to the story →

Result
Streak-up animation + seal + breakdown.

Store

Premium stories, yours to keep

Buy a story and it's added to your Home instantly.

Dracula

A Gothic horror, inspired by Bram Stoker

Trade letters and warnings as a count's shadow falls over London.

5 scenes

Buy \$3.99

Frankenstein

A science tragedy, inspired by Mary Shelley

Give a voice to the creature everyone else refuses to hear.

5 scenes

Buy \$3.99

Pride and Prejudice

A romance of manners, inspired by Jane Austen

Win the conversation — and maybe more — one sharp reply at a time.

5 scenes

Buy \$2.99

Store
Premium books, Buy → library.

Leaderboard

See how you stack up

#14

AA

Shirsho

Avg 78 · You

Avg Score

Total Score

Clarity

Grammar

Tintin

Avg Score 95 · Total 7,793

95

Hermione

Avg Score 93 · Total 7,510

93

Naruto

Avg Score 91 · Total 7,120

91

Mulan

Avg Score 90 · Total 6,980

90

Kabir

Avg Score 89 · Total 6,640

89

Sora

Avg Score 88 · Total 6,210

88

Diego

Avg Score 85 · Total 5,870

85

Ananya

Avg Score 84 · Total 5,540

84

Leo

Avg Score 83 · Total 5,210

83

Leaderboard
Pinned rank, filter chips, medals.

Profile

AA

Shirsho

Joined 24 days ago

2-day streak

M T W T F S S

5

Scenes

2h 40m

Duration

78

Avg score

#14

Rank

Skill breakdown

Relevance

82

Smoothness

64

Clarity

78

Profile
Stats, skills, coaching cards.

Plans

Power up your learning

CURRENT PLAN

Free

Energy-based learning

1/4 energy

Resets every 12 hours

Full energy completes 4 scenes

Access up to 10 books on Free, unlimited on Pro

Books accessed

2/10

Amiboli Pro

\$6.99 /month

Unlimited energy

Unlimited books

Priority new stories

Detailed skill analytics

Upgrade to Pro

Free vs Pro

Plans
Free vs Pro.

Build target: Flutter mobile app + FastAPI backend. **This document** describes the existing React/Vite prototype in full so a developer can rebuild it as a **production, backend-driven Flutter app**. Special emphasis is placed on **the Scene play engine** (§9) — it is the heart of the product.

The living prototype is the visual source of truth. Deployed: <https://amiboli-app-prototype.vercel.app> — open it in mobile Chrome (or DevTools at 390x844). Every screen referenced below is reachable at the route listed in §8. When a layout detail is ambiguous in text, open that route and look. The whole prototype is hardcoded/in-memory with **no backend** — your job is to reproduce the same UX while persisting everything in FastAPI.

1. Product overview

Amiboli teaches **spoken English by playing through interactive stories**. Famous books (public-domain or "inspired by") are split into **scenes**. A scene plays like a tiny radio drama: narration and an NPC speak; at key moments the app hands the conversation to the user, who **speaks a line out loud**. The app scores the speech on four skills, the NPC reacts in character, and the user earns XP, keeps a streak, climbs a leaderboard, and spends "energy" to play.

The pitch: you don't study English, you *act in a story* and speak to move it forward.

Three pillars: **Stories first · Speaking out loud · Delightfully simple.**

2. Target stack & recommended Flutter packages

Concern	Prototype (React)	Flutter recommendation
Navigation	React Router (hash routes)	<code>go_router</code>
State	React context (Energy, Library) + local	<code>riverpod</code> (or <code>provider</code>)
Networking	none (in-memory)	<code>dio</code> + <code>retrofit</code> (typed)
Vector art (mascot)	inline SVG <code></code>	<code>flutter_svg</code> (asset SVGs)
Scenery / characters	hand-drawn inline SVG	<code>flutter_svg</code> or <code>CustomPainter</code> (see §10)
Animations	Framer Motion	<code>flutter_animate</code> , <code>Animated*</code> widgets, <code>AnimationController</code>
Microphone capture	Web Audio <code>getUserMedia</code>	<code>record</code> (capture)
Live waveform	canvas from analyser	<code>record</code> 's amplitude stream → custom painter, or <code>audio_waveforms</code>
NPC voice (TTS)	Web Speech Synthesis	<code>flutter_tts</code>
(optional) real STT	Web Speech Recognition	<code>speech_to_text</code> — display only; scores come from backend
Sound FX (chime)	Web Audio oscillator	<code>just_audio</code> / <code>audioplayers</code> with a tiny asset, or <code>soundpool</code>
Fonts	Nunito + Lora (Google)	<code>google_fonts</code> (Nunito, Lora)
Confetti / sparkles	Framer	<code>confetti</code> or custom
Local persistence/cache	none	<code>hive</code> / <code>shared_preferences</code>

Backend: FastAPI + a DB (Postgres recommended). Stores users, books, scenes (with beats), progress, scores, energy/streak ledger, leaderboard, store/purchases. The one piece the prototype *fakes* and the backend must make real is **speech scoring** (§9.7).

3. Design system (brand guidelines v2)

Colour (each colour has a fixed role — never interchange)

Token	Hex	Role
Story Violet	#4A35B0	Primary — nav, buttons, key surfaces
Violet Lavender	#7C5CDB	hover / lighter primary
Violet Whisper	#EEEEAF	tints, chips, light fills
Ami Teal	#2ABFA8	secondary, progress, mascot
Teal dark	#1D9E86	teal pressed
XP Gold	#FFD166	rewards, streaks, XP
Gold dark	#E8920A	gold accents
Coral	#D85A30	errors / destructive / accents
Ink	#1A1040	primary text & dark surfaces
Ink soft	#5A4D8A	secondary text
Ink mute	#9D90D8	muted text/labels
App background	#F6F4FF	light app background
Border	#E2DEFF	hairline borders
Forest Green	#2E7A48 / Slate Blue #1E3A6E	supporting

Accessibility checked: white on #4A35B0 = 7.2:1 (AAA); #1A1040 on #FFD166 = 9.1:1.

Typography

- Nunito (700/800/900) — all UI, headings, buttons, labels.
- Lora (400/600 + italic) — story prose, recaps, narration, definitions, scene dialogue. Never swap the two.

Radius & spacing tokens

- Radius: chip 4, button 8, card 14, sheet 22, frame 32 (px).
- Spacing scale (4px base): 4, 8, 16, 24, 32, 48.

Mascot — "Ami" (single teal book)

- One character, 7 moods: happy, sad, excited, celebrating, thinking, talking, sleeping.
- Flat art, no gradients/glows/shadows/recolour/stretch. Files: assets/mascots/book-teal/book-<mood>.svg (in prototype: public/mascots/book-teal/).
- Used sparingly (deliberate product decision): as the app logo and on a few static screens only — Splash, Auth, Onboarding, the Home "more stories" end card. Do NOT sprinkle it across functional screens.

Layout frame

Mobile-first, designed for 360–430px width. On large screens the prototype centers the app in a phone frame; on a real device it's full-screen. Respect safe areas (notch / bottom inset).

4. App map & navigation

Bottom tab bar (icons only, active tab shows a violet dot): Home 🏠 · Store 📚 · Ranks 🏆 · Profile 🐼 · Plans ⚡ . Settings is reached from the avatar in the Home top-right corner (not a tab).

Splash —auto 2s→ Home
Onboarding (3 slides, skippable) ▶ Auth (UI only) ▶ Home
Home (book feed) —tap a book→ Book screen —"Continue / Begin"→ Scene player
Scene player —complete→ Result (score + streak) → Book / Leaderboard
Scene player —leave early (x)→ Ad → Book
Energy pill → Energy sheet —"watch ad +1s"→ Ad → back
Tabs: Home · Store · Ranks(Leaderboard) · Profile · Plans ; Settings via avatar

Routes used in the prototype (mirror as go_router paths):

Screen	Route	Tabbed?
Splash	/	no
Onboarding	/onboarding	no
Auth	/auth	no
Home (book feed)	/home	yes
Book detail	/book/:bookId	no (full-screen)
Scene player	/play/:bookId/:sceneId	no
Result / score	/score/:bookId/:sceneId	no
Interstitial ad	/ad/:bookId/:sceneId	no
Store	/store	yes
Leaderboard	/leaderboard	yes
Profile	/profile	yes
Plans	/plans	yes
Settings	/settings	yes (no tab; via avatar)

5. Core mechanics & formulas

Skills (4, every score is 0–100): `relevance`, `smoothness`, `clarity`, `grammar`. - relevance = did the line fit the moment/prompt; smoothness = flow vs pauses; clarity = pronunciation/shaping; grammar = sentence correctness.

Per-scene aggregation (used on the Result screen): - For each skill: average that skill across all `user_turn`s in the scene. - `sceneAvg = round(mean over turns of (mean of the 4 skill scores))`. - `pointsEarned (XP) = sceneAvg * 4`. - `newTotalScore = user.totalScore + pointsEarned`.

Streak: completing a scene increments the day-streak by 1 (prototype animates 2 → 3). Backend should implement real daily-streak logic (consecutive active days).

Energy: - `max = 4`, each scene costs **1** energy to play. - Refills over time (copy says "every 12 hours"); implement a regen schedule. - **Watch-an-ad → +1 energy**, available **any time** (no cooldown), capped at max. - Pro plan = unlimited energy.

Big moment: a turn is a "big moment" if authored `isBigMoment` **or** any skill ≥ 90 → triggers brighten + chime + golden chip on the stage.

Leaderboard: ~15 users; sortable by `avg`, `total`, `clarity`, `grammar`, `smoothness`, `relevance`. The current user is pinned at top with their rank and highlighted in the list. Top 3 get medals.

Plans: - **Free** — `4 bolts / 12h`, "full energy completes 4 scenes", access up to **10 books**. - **Pro** — `$6.99/mo`, unlimited energy, unlimited books, priority new stories, detailed analytics. ("Upgrade" is non-functional in prototype.)

Store / Library: premium books have a `price` + `tag` (Bestseller/New/...). "Buying" adds the book to the user's **library**, after which it appears in the Home feed and is openable. (Prototype keeps this in memory; backend persists purchases/entitlements.)

Ads: the **only** place ads appear is the **energy top-up** flow (watch → +1⚡) and when **leaving a scene early**. There is **no** ad after completing a scene. (This changed across iterations — keep ads minimal.)

6. Data model (backend entities)

Below are the shapes the prototype uses as hardcoded data. Promote them to DB tables / API resources. Field names can stay; add ids, timestamps, ownership.

6.1 User

```
{
  "id": "u_...",
  "name": "Shirsho",
  "initials": "SH", // for avatar
  "avatarColor": "#4A35B0",
  "joinedAt": "2026-05-...",
  "plan": "free", // free | pro
  "energy": { "current": 1, "max": 4, "refillsEvery": "12h", "nextRefillAt": "..."},
  "streak": 2, // consecutive-day streak
  "weeklyActivity": [true,true,false,true,true,false,false], // Mon-Sun
  "scenesCompleted": 5,
  "totalSpeakingTime": "2h 40m",
  "rank": 14,
  "scores": { "relevance": 82, "smoothness": 64, "clarity": 78, "grammar": 88 }, // running profile averages
  "avgScore": 78,
  "totalScore": 3920
}
```

6.2 Book

```
{
  "id": "scarlet",
  "title": "A Study in Scarlet",
  "subtitle": "A Sherlock Holmes story", // author-style line
  "pitch": "Solve a murder alongside the world's sharpest detective.",
  "themeId": "victorian-london", // → visual world ($9.9 / bookThemes)
  "coverFigure": "detective", // Home-banner character ($10)
  "recapText": "Holmes hands you the telegram...", // shown when in-progress
  "introText": "You arrive at 221B Baker Street...", // shown when not started
  "isPremium": false,
  "price": null, "tag": null, "coverEmoji": null, // premium-only fields
  "scenes": [ Scene... ]
}
```

6.3 Scene (list item inside a Book — the map node, NOT the playable beats)

```
{
  "id": "sc5",
  "sceneName": "The Cabman's Tale",
  "status": "completed | current | locked", // per-user progress
  "score": 84, // present when completed (the sealed score)
  "narrationAfter": "A cab rattles through the fog..." // 1 line shown between sealed scenes
}
```

Progress rules: a book is "started" if any scene is `completed`. The first not-completed scene is `current`; everything after is `locked` (its name is hidden in "fog" on the Book screen — future scene names are spoilers).

6.4 PlayableScene (the beats — see §9; lives separately, keyed by bookId+sceneId)

Full schema and the canonical example are in §9.2 / §9.3.

6.5 LeaderboardEntry

```
{ "name": "Tintin", "initials": "TI", "color": "#EF4444",
  "relevance": 97, "smoothness": 92, "clarity": 96, "grammar": 95,
  "avg": 95, "total": 7793, "isMe": false }
```

6.6 Plan / StoreBook / Notification

- Plan: { free:{energy, rules[], booksCap}, pro:{price, perks[], comparison[] } }.
- StoreBook = Book + { price:"\$3.99", priceValue:3.99, tag:"Bestseller", coverEmoji:"👉" }.
- Notification: { id, emoji, title, body, time, urgent? }.

7. Suggested REST API (FastAPI)

Minimum surface to drive the app. Adjust to taste; keep responses matching §6.

```

POST /auth/login          → { token, user }
GET /me                   → User (energy, streak, scores, rank...)
GET /books                → [Book] (free + owned), per-user progress baked in
GET /books/{bookId}       → Book (with scenes[] + recap/intro)
GET /store                → [StoreBook] (not-owned premium)
POST /store/{bookId}/purchase → { ok, book } (entitlement; payment later)

GET /play/{bookId}/{sceneId} → PlayableScene (beats WITHOUT `simulated` scores)
POST /play/{bookId}/{sceneId}/turn
  body: { beatIndex, audio(file)|transcript }
        → { transcript, scores:{relevance,smoothness,clarity,grammar},
            npcReaction, mood, isBigMoment } ← the real version of `simulated`
POST /play/{bookId}/{sceneId}/complete
  body: { perTurnScores[], elapsedSec }
        → { sceneAvg, pointsEarned, newTotalScore, newStreak, energyAfter, sealedScore }

POST /energy/watch-ad     → { energy } (+1, capped at max)
GET /leaderboard?sort=avg → { me:Entry, entries:[Entry] }
GET /notifications        → [Notification]
GET /plans                → Plan config
POST /plans/upgrade       → checkout/session (later)

```

Key backend responsibility — `/turn` : receives the user's recorded audio for a `user_turn` beat and returns the same shape the prototype hardcodes in `simulated` (§9.7). This is where ASR + scoring/LLM live.

8. Screen-by-screen spec

For each: open the live route to see it. Layouts described top→bottom.

- Splash** `/` — violet gradient, animated **Ami (book) logo** + "Amiboli" + "Learn English through stories" + loading dots. Auto-advances to Home after ~2s.
- Onboarding** `/onboarding` — 3 swipeable slides (Ami mascot per slide, big), title + body, page dots, "Next/Get started", **Skip**. Slides: *Live the story* · *Speak to move forward* · *Get scored & climb*.
- Auth** `/auth` — Ami logo, "Welcome back", email/password fields (no validation), **Continue** (violet) → Home, divider, **Continue with Google** → Home, "Create account". UI only.
- Home — book feed** `/home` — vertical **scroll-snap** feed, **one book per screen**. Each book section: - Full-bleed **themed world** background (atmospheric, code-drawn — §9.9). - A **big, gently-moving silhouette character** for the book (detective / commuter / child / ... — §10). - Bottom frosted panel: **subtitle** (eyebrow) → **title (Lora)** → pitch → progress (`4/6 scenes completed` · Avg 80 or Not started yet) → a row of **scene dots** (filled-with-score / glowing current / dim future) → primary button **Continue/Start this story**. - Top bar floats over all sections: **energy pill** (single ⚡ + `1/4` + " ", tap → energy sheet) · **streak** (🔥 2) · **bell** (notifications sheet) · **avatar** (→ Settings). - Right-edge page-dot indicator. Final section: "Want more stories?" + Ami + **Open the Store**.
- Book detail** `/book/:id` — back button + title; **StoryCard** hero (recap "Previously, in ..." if in progress, or intro "Your story begins / You arrive as..." if not) with primary CTA (**Continue** · **Scene N / Begin Scene 1**); then a **vertical scene path**: completed = wax-seal stamp w/ score + a line of `narrationAfter` between scenes; current = glowing "You are here" card (`uses 1 ⚡`); locked = "Hidden in the fog...". Tapping the current playable scene → `/play/...` ; other taps → toast.
- Scene player** `/play/:bookId/:sceneId` — see §9 (the core).
- Result/score** `/score/:bookId/:sceneId` — **streak-up celebration** (flame, count rolls e.g. 2→3, sparks, week dots) → wax seal w/ `sceneAvg` "Scene sealed" → quick facts (time · `-1 ⚡` · turns) → **Score earned** (`+320` , total `3,920` → `4,240` , streak) → **How you spoke** (4 skill bars with `Δ vs your avg`) → **Best line** (the big-moment quote + reaction) → coaching tip (weakest skill) → **rank nudge** card (→ leaderboard) → sticky **Replay / Back to the story**.
- Ad** `/ad/:bookId/:sceneId` — mock interstitial: "Advertisement", a fake sponsor card, 5s skip countdown, "Tired of ads? Go Pro" (→ Plans), **Continue**. Destination passed in (energy ad → back; leave-early → book).
- Store** `/store` — list of premium book cards: themed cover (emoji), title, subtitle, pitch, corner **tag**, **Buy \$X** → adds to library (toast) → card flips to "✓ In library / Open".
- Leaderboard** `/leaderboard` — pinned **my-rank** card (#14, avatar, avg) → filter chips (Avg/Total/Clarity/Grammar/Smoothness/Relevance) that re-sort → ranked list (top-3 medals, me highlighted).
- Profile** `/profile` — identity card (avatar, name, joined, weekly **streak calendar**) → stats grid (Scenes, Duration, Avg, Rank) → **skill bars** → **Strengths / Areas to grow / Suggestions** coach cards → "Edit personal info".
- Plans** `/plans` — current **Free** card (energy state, rules, "Books accessed 2/10" progress) → **Pro** gradient card ("MOST POPULAR", perks, **Upgrade to Pro**) → Free-vs-Pro comparison table.
- Settings** `/settings` — grouped rows: Account, Notifications toggle, Sound toggle, Language, Privacy, Terms; **Log out** → Auth; app version.

Transient sheets: **Notifications** (bottom sheet, list of items) and **Energy** (top sheet: bolts, "Each scene costs 1 ⚡ ", **Watch an ad for +1 ⚡** , or "Energy full").

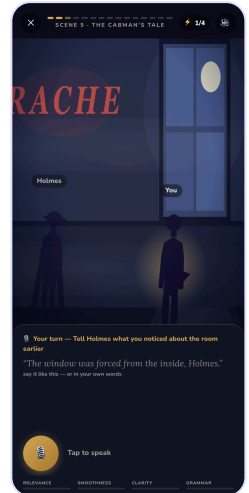
9. ★ THE SCENE ENGINE (build this carefully)

The scene player is a **generic beat player**: it renders whatever the scene data says, with **zero scene-specific code**. Everything below is data-driven. Reproduce this engine in Flutter; author scenes as JSON served by the backend.

9.1 Concept — "The Stage"

One **fixed three-zone layout** that never changes during a scene (no modals, no route changes):

HEADER: <code>x(leave→confirm) · beat progress bar · "Scene 5 · Title" · ⏪ · 🎧</code>	← 1 segment per beat
STAGE (shadow-theatre): themed SVG set + props + lighting + 2 characters (NPC + You) + fog	← themed set, 2 silhouette characters posing, camera pans/zooms inside a fixed overflow-hidden viewport
CAPTION PANEL (fixed height ~310px): story / mic / feedback / summary + 4 thin skill meters at the bottom	← morphs between 4 modes, NEVER resizes (so the stage above never shifts)



The scene player — the fixed 3-zone layout (header · stage · caption in mic mode).

Critical layout rule: the caption panel is a **fixed height**; it *morphs* its contents (cross-fade ~220ms) but never changes size, so the stage above never reflows. In Flutter: a fixed-height container with an `AnimatedSwitcher`.

9.2 Beat data format (the authoring schema)

A scene is `{ id, bookId, sceneNumber, title, setId, npc:{name}, beats:[...] }`. Each beat is one of three types:

```
// 1) Narration – narrator prose (typed out, optionally voiced)
{ "type": "narration", "text": "...", "stage": { ... }? }

// 2) NPC line – a character speaks (typed out, voiced)
{ "type": "npc_line", "speaker": "Holmes", "text": "...", "mood": "neutral?", "stage": { ... }? }

// 3) User turn – the user must SPEAK
{ "type": "user_turn",
  "intent": "Tell Holmes what you noticed about the room", // the prompt shown
  "suggestedLine": "The window was forced from the inside, Holmes.", // ghost guide
  "stage": { ... }?
  // In the PROTOTYPE only, the result is authored inline as `simulated`.
  // In PRODUCTION, omit `simulated`; the backend returns it from /turn ($9.7).
  "simulated": {
    "transcript": "I think the window was forced from the inside, Holmes.",
    "scores": { "relevance":78, "smoothness":72, "clarity":75, "grammar":81 },
    "npcReaction": "Holmes nods slowly.",
    "mood": "neutral", // impressed | neutral | puzzled
    "isBigMoment": false
  }
}
```

stage (optional on any beat — directs the stage at beat start; omitted fields keep current state, unknown names no-op):

```
"stage": {
  "camera": "wide|holmes|you|constable>window|wallWriting|lamp|door",
  "highlight": "window|wallWriting|lamp|door", // ~2s pulse on a prop
  "lighting": "calm|tense|revelation",
  "poses": { "holmes": "speaking",
    "you": { "pose": "pointing", "target": "right" } },
  "enter": "constable", // a character walks onto the stage
  "exit": "constable" // ...or off it
}
```

Poses: `idle · speaking · pointing(+target left|right|up) · impressed · puzzled`.

9.3 Canonical scene (ship this exact scene; it's the reference)

This is **A Study in Scarlet · Scene 5 · "The Cabman's Tale"** — 16 beats, 4 user turns, with a deliberate score arc (decent → shaky → good → **big moment 95+**). Use it verbatim to validate your player and your scoring contract.

```
{
  "id": "sc5", "bookId": "scarlet", "sceneNumber": 5,
  "title": "The Cabman's Tale", "setId": "lauriston-room", "npc": { "name": "Holmes" },
  "beats": [
    { "type": "narration", "text": "Night has fallen over Lauriston Gardens when Holmes leads you back into the empty house. The word on the wall waits in the dark.", "stage": { "camera": "wide", "lighting": "calm" } },
    { "type": "npc_line", "speaker": "Holmes", "text": "The daylight showed us what the killer wanted seen. The dark will show us what he hoped to hide.", "stage": { "camera": "wallWriting", "highlight": "wallWriting", "poses": { "holmes": { "pose": "pointing", "target": "left" } } },
    { "type": "user_turn", "intent": "Tell Holmes what you noticed about the room earlier", "suggestedLine": "The window was forced from the inside, Holmes.",
      "simulated": { "transcript": "I think the window was forced from the inside, Holmes.", "scores": { "relevance": 78, "smoothness": 72, "clarity": 75, "grammar": 81, "npcReaction": "Holmes nods slowly.", "mood": "neutral" } },
    { "type": "npc_line", "speaker": "Holmes", "text": "From the inside. Curious, is it not — a house locked in fear, yet opened from within.", "stage": { "camera": "window", "highlight": "window", "poses": { "holmes": { "pose": "pointing", "target": "right" } } },
    { "type": "narration", "text": "Holmes crosses to the cold hearth and runs one gloved finger through the ash.", "stage": { "camera": "wide", "poses": { "holmes": "idle" } },
    { "type": "user_turn", "intent": "Ask Holmes what he has found in the ash", "suggestedLine": "Is there something in the ashes, Holmes?",
      "simulated": { "transcript": "Is there... something... in the ash, Holmes?", "scores": { "relevance": 70, "smoothness": 58, "clarity": 66, "grammar": 74, "npcReaction": "Holmes glances up, puzzled by your hesitation.", "mood": "puzzled" },
      "stage": { "lighting": "tense" } },
    { "type": "npc_line", "speaker": "Holmes", "text": "Ash from a cigar no London shop sells. Our man brought his habits from somewhere far away.", "stage": { "lighting": "calm", "camera": "holmes", "poses": { "holmes": "speaking" } },
    { "type": "npc_line", "speaker": "Constable", "text": "Begging your pardon, sirs — there was a cab stood waiting out front that night, near midnight.", "stage": { "enter": "constable", "camera": "door", "poses": { "constable": "speaking", "holmes": "idle" } },
    { "type": "narration", "text": "The constable touches his helmet and returns to his post by the gate.", "stage": { "exit": "constable", "camera": "wide" },
    { "type": "user_turn", "intent": "Confirm it — tell Holmes what you saw by the curb", "suggestedLine": "There were two wheel ruts by the curb — a cab stood waiting here.",
      "simulated": { "transcript": "There were two wheel ruts by the curb — a cab stood waiting here.", "scores": { "relevance": 84, "smoothness": 79, "clarity": 82, "grammar": 86, "npcReaction": "Holmes's whole posture brightens.", "mood": "impressed" } },
    { "type": "npc_line", "speaker": "Holmes", "text": "Precisely. The killer did not walk to this house — he was driven to it, and he was not alone.", "stage": { "camera": "holmes", "poses": { "holmes": "speaking" } },
    { "type": "narration", "text": "Holmes turns to you, the gaslight catching the edge of a rare smile.", "stage": { "poses": { "holmes": "impressed" } },
    { "type": "user_turn", "intent": "Put it together — tell Holmes who you should be looking for", "suggestedLine": "Then we're not looking for a passenger, Holmes — we're looking for the cabman.",
      "simulated": { "transcript": "Then we're not looking for a passenger, Holmes — we're looking for the cabman himself.", "scores": { "relevance": 95, "smoothness": 88, "clarity": 91, "grammar": 93, "npcReaction": "Holmes laughs — actually laughs — and claps you on the shoulder.", "mood": "impressed", "isBigMoment": true } },
    { "type": "npc_line", "speaker": "Holmes", "text": "There are days, my friend, when you astonish me. Tomorrow, we hunt a cabman.", "stage": { "lighting": "revelation", "camera": "holmes", "poses": { "holmes": "impressed" } },
    { "type": "narration", "text": "The fog folds itself over the empty house behind you, and the night swallows the street whole.", "stage": { "lighting": "calm", "camera": "wide", "poses": { "holmes": "idle", "you": "idle" } }
  ]
}
```

9.4 Beat sequencing & timing (constants from the prototype)

- **AUTO_ADVANCE_MS = 1500** — after a story beat's typewriter finishes, auto-advance after 1.5s if the user does nothing.
- **LISTENING_MS = 1000** — after the user stops recording, show a "listening..." shimmer ~1s before revealing the result.
- **FEEDBACK_MS = 2500** — the feedback moment lasts ~2.5s, then auto-advance.
- **PANEL_H = 310** — caption panel fixed height (px).

Per-beat lifecycle (engine): on entering beat *i*: 1. Apply its **stage** directions (camera/lighting/poses/highlight/enter/exit) **plus auto-direction**: NPC speaker auto-poses **speaking**; on a **user_turn** the camera auto-drifts to **you**; after a turn, re-pose the NPC from the result **mood** (**impressed-impressed**, **puzzled-puzzled**, else **speaking**); a big moment forces **lighting: 'revelation'**. 2. If story beat → start typewriter; if not muted, speak the text via TTS. 3. If **user_turn** → switch caption to **mic mode**.

9.5 Story mode (narration / npc_line)

- Caption shows speaker label (**HOLMES** / **NARRATOR**, Lora-styled) + the text via **typewriter** (~26ms/char).
- **Tap the caption once** → instantly finish the typing; **tap again** → advance to next beat. Auto-advance 1.5s after typing completes.
- **TTS**: speak NPC/narration lines with a British-leaning English voice; **mute toggle** in header; if TTS unavailable, silently skip (pacing comes from the typewriter, not audio).
- Stage: active speaker is lit (rim glow + slight scale up), the other dims; head does a subtle talk-bob while typing.

9.6 Mic mode (the user's turn) — real microphone

When a `user_turn` begins: - Stage **dims slightly** and the **user's character lights up** — make it unmistakable that it's the user's turn (clarity beats subtlety). - Caption shows: the **intent** as a colored prompt ("👤 Your turn — ..."), the **suggestedLine** as faint ghost serif text, and a hint "say it like this — or in your own words". - A **mic button + live waveform + timer** (15s cap). - **Real capture**: request mic permission; record real audio; render a **live waveform from real input amplitude**. This realness is core to the feel. - **Fallback (required)**: if permission denied / unavailable (e.g. no mic), switch to **"Hold to speak"** — holding the button animates a fake waveform. The whole loop must be testable without a mic. - On stop (tap again, or 15s, or release in hold-mode): show **"listening..."** shimmer ~1s, then transition to **feedback mode**.

In the prototype scores are read from `simulated`. **In production**: upload the recorded audio to `POST /play/.../turn` and use the returned object (§9.7). (Optionally also run on-device STT for a *display-only* transcript; the authoritative transcript+scores come from the backend.)

9.7 Scoring contract (prototype `simulated` ⇒ backend `/turn` response)

The backend must return exactly this for each user turn:

```
{
  "transcript": "what the learner actually said (ASR)",
  "scores": { "relevance":0-100, "smoothness":0-100, "clarity":0-100, "grammar":0-100 },
  "npcReaction": "Holmes nods slowly.", // 1 short in-character line
  "mood": "impressed | neutral | puzzled", // drives the NPC pose + styling
  "isBigMoment": false // optional; also true if any score ≥ 90
}
```

- `relevance` ← semantic match of transcript to the beat's `intent` /expected content.
- `smoothness` ← fluency/pauses/fillers (timing + ASR confidence/pauses).
- `clarity` ← pronunciation/intelligibility (ASR acoustic confidence / phoneme scoring).
- `grammar` ← grammatical correctness of the transcript (LLM/grammar model).
- `npcReaction` + `mood` ← an LLM prompt conditioned on the scene, the NPC, the intent, and the scores (warm, in-character, never shaming). Authoring can also provide reaction *templates* per score band if you want determinism.

To author a scene, the writer supplies, per `user_turn`: `intent`, `suggestedLine`, and ideally a short **reference/expected answer** and **NPC reaction templates** for high/medium/low bands. The runtime fills `transcript/scores/mood/isBigMoment`.

9.8 Feedback mode + persistent scoring layer

- **Persistent (always on during the scene)**: four thin **skill meters** at the bottom edge of the caption panel showing the **running averages** (no numbers) — they tick up smoothly after each turn.
- After a turn (the ~2.5s feedback window):
- **NPC reacts on stage** = the PRIMARY feedback channel: the reaction line appears under the NPC, with mood styling (impressed = warm glow toward the user; puzzled = slight dim + head tilt).
- **One score chip floats up** from the user's character: strongest skill if it shines (`+91 grammar`, golden if ≥ 90) else the weakest (`smoothness 58`).
- The caption shows "You said: "".
- **Big moment** (`isBigMoment` or any skill ≥ 90): brief celebration — stage brightens (`revelation`), a **golden chip**, a soft **two-note chime** (Web Audio in prototype → a tiny asset/synth in Flutter). Keep it < 1.5s — a spark, not fireworks.

9.9 The shadow-theatre stage (sets, props, camera, lighting, characters)

Everything is **drawn in code** (flat fills, layered tones, fake glows from stacked translucent shapes — **no image files, no blur filters**; animate transforms/opacity only for 60fps).

- **Set** (`setId`, e.g. `lauriston-room`): a world drawn ~130%×118% of the stage viewport so the camera can move within a fixed, `overflow:hidden` viewport. Scene 5's room: two-tone walls, a moonlit window (panes + moon), a wall gas-lamp with warm glow, the scrawled word **"RACHE"** in glowing serif, a door, floorboards with light pooling, 1–2 drifting fog wisps (the one ambient), a vignette.
- **Named camera targets** (transform the world; ~1.5s slow ease, one move per beat max): `wide` (default), each character (`holmes`, `you`, `constable`), each prop (`window`, `wallWriting`, `lamp`, `door`). Example values (world-space px): `wide {x:0,y:0,scale:1}`, `holmes {x:46,y:26,scale:1.22}`, `window {x:-96,y:44,scale:1.3}`.
- **Prop highlight**: brighten + gentle pulse + larger glow for ~2s.
- **Lighting moods** (full-stage tint overlays, crossfade): `calm` (default, warm 0.05), `tense` (cool 0.4 — dimmer/cooler), `revelation` (warm 0.2 — brighter).
- **Character rig**: a layered silhouette (near-black `#0F0D2E`, **no faces** — identity from shape: hats, coats, props). Groups: back-arm / body / head / front-arm, each with a sensible pivot, animated between **poses** with spring physics (~400ms). Idle life always on: breathing (scaleY 1↔1.012, offset per character), occasional head turns, talk-bob while speaking. Active speaker gets a rim glow; inactive dims. Characters **walk on/off** for `enter` / `exit`.

- **Scene 5 cast:** Holmes (tall, long coat, deerstalker + pipe), You/Watson (bowler, shorter coat), Constable (custodian helmet) — appears for one beat via `enter`.
- **Guardrails:** at most ONE camera move OR one highlight pulse active at a time; idle life + fog are the only constant motion; if it ever feels busy, cut motion.

9.10 End of scene → Result

When the last beat finishes, the caption morphs to a brief **"Scene complete!"** (seal stamp + Replay/Continue). **Continue → Result screen** (`/score/...`, §8.7) which leads with the **streak-up animation**, then the full breakdown. One energy bolt drains in the header on completion. (No ad after completing.)

9.11 Authoring a new scene (hand to your writers)

1. Pick the book → its `themeId` (world) and `setId` (which set to draw).
2. Write 12–16 beats alternating narration / `npc_line` / **3–5 user_turns**, with a score **arc** (don't make every turn perfect).
3. Each `user_turn`: write `intent` (what the user should accomplish) + `suggestedLine` + (production) a reference answer + NPC reaction templates per band.
4. Add `stage` directions to ~6+ beats: camera to whatever matters, `highlight` a prop when it's named, `lighting:'tense'` on a stumble, `'revelation'` on the win, poses (`pointing` at clues, `impressed` / `puzzled` reactions), `enter` / `exit` for extra characters.
5. All text **original** — evocative of the book, never quoting copyrighted novels (titles are fine).

10. Assets & how to port the art to Flutter

- **Mascot (Ami):** 7 flat SVGs — ship as Flutter assets (`assets/mascots/book-teal/book-<mood>.svg`) and render with `flutter_svg`. Use only as logo + the few static screens.
- **Home book-banner characters:** flat **silhouette** figures drawn in code (prototype: `src/components/story/BookCharacter.jsx`). Figures: `detective`, `commuter`, `child`, `lady`, `caped`, `reader` (default). Each book maps to one (`coverFigure`). A soft theme-glow sits behind so the dark silhouette reads on both dark and light worlds. Port as `CustomPainter` s or pre-exported SVGs; animate a slow sway ($\pm 1.4^\circ$) + breathe.
- **Book worlds / scenery:** code-drawn layered SVG per `themeId` (prototype: `src/components/story/Scenery.jsx`): `victorian-london` (navy night, rooftops, gas lamps, fog), `rail-dusk` (slate/violet, rails, signal, passing light streaks), `southern-summer` (cream/amber, oak tree, porch, fireflies). Each has exactly **one ambient effect**.
- **Stage set + characters:** code-drawn (prototype: `src/components/play/Stage.jsx`, `Character.jsx`). Port via `CustomPainter`.
- **No raster images** anywhere except the mascot SVGs. The PWA launcher icon is a separate asset (`public/icon.svg`) — not part of in-app UI.

Practical Flutter path: keep the scenery/characters as **SVG assets exported from the prototype** (simplest fidelity) OR re-implement as `CustomPainter` (smaller, animatable per-layer). The mascot stays SVG.

11. Motion & "feel" checklist

- Smooth screen transitions; tappable elements get press/scale feedback.
- Typewriter dialogue; the caption panel **morphs without resizing**.
- Stage: slow film-drift camera, springy poses, subtle breathing/fog. **Stillness is part of the craft** — never more than one big motion at once.
- Result: the streak count **rolls** with a bounce; sparks; meters tick.
- Big moment: stage brighten + golden chip + soft chime, < 1.5s.
- Target **60fps on mid-range phones** — transforms/opacity only; avoid heavy blur and animating layout.

12. Prototype constraints to REMOVE in production (they're fake here)

- No backend / persistence — everything resets on refresh. **You** add FastAPI + DB.
- **Scoring is simulated** (authored per line). Replace with real ASR + scoring at `POST /turn`.
- Auth is UI-only (any tap → Home). Implement real auth.
- "Buy", "Upgrade to Pro", ads, "Log out" are non-functional toasts/mocks. Wire to real entitlements/payments/ad SDK.
- Energy/streak/XP are in-memory; persist + add server-side regen/anti-cheat.

13. Suggested build order

1. **Design system** (colors, Nunito/Lora, radii, buttons, the phone frame, tab bar).
2. **Auth** + `/me` and the **Home book feed** (themed worlds + banner characters + scene dots).
3. **Book screen** (recap/intro + scene path + progress).
4. 🌟 **Scene engine** (§9) against the canonical Scene 5 with **simulated scores first** (port the `simulated` blocks), so you can finish the whole UX before the AI exists.
5. **Result** → **streak/XP/energy** flow + **leaderboard**.
6. Replace simulated scoring with the real `/turn` (ASR + scoring).
7. **Store/library**, **Plans**, **Settings**, **energy ad** loop, notifications.
8. Polish motion & 60fps pass.

Live reference: <https://amiboli-app-prototype.vercel.app> · prototype source in this repo (`src/` — see `src/data/scenes.js` , `src/screens/Play.jsx` , `src/components/play/*` , `src/data/bookThemes.js` , `src/data/dummyData.js`).